

УНИВЕРСИТЕТ ИТМО

Факультет прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Дисциплина «Проектная документация»



ОТЧЕТ по

лабораторной работе Курсовая работа по предмету ”Мобильная разработка”

“Мобильное приложение для отображения ленты новостей”

Выполнил:

Таипов Тимур Алексеевич

Поток:

Группа К3441

Преподаватель:

Шатинский Григорий Сергеевич

Санкт-Петербург

2026 г.

Содержание

ВВЕДЕНИЕ	5
1 Обзор проекта	5
1.1 Назначение приложения	5
1.2 Контекст и ограничения	5
1.3 Целевая аудитория	6
1.4 Ожидаемые результаты	6
1.5 Технологический стек	6
2 Требования и постановка задачи	6
2.1 Функциональные требования	6
2.2 Нефункциональные требования	7
2.3 Ограничения и допущения	7
2.4 Сценарии использования	7
3 Анализ предметной области	7
3.1 Типовые приложения класса	7
3.2 Пользовательские ожидания	8
3.3 Выводы для проектирования	8
4 Архитектура	8
4.1 Общая схема	8
4.2 UI слой	8
4.3 ViewModel слой	8
4.4 Repository слой	9
4.5 Data слой	9
4.6 MVVM и поток данных	9
4.7 Сетевой слой	9
4.8 Локальное хранилище	10
4.9 Репозиторий как единая точка доступа	10
4.10 Фоновая синхронизация	11
4.11 Уведомления	11
4.12 Онлайн/оффлайн режим	12

5	Проектирование и моделирование	12
5.1	Последовательность обновления	12
5.2	Локальные состояния	12
6	Пользовательский интерфейс	12
6.1	Навигация	12
6.2	Компоненты Material Design	13
6.3	Темизация и стили	13
6.4	Лента сообщений	13
6.5	Карточка сообщения	13
6.6	Профиль	15
6.7	Настройки	15
7	Руководство пользователя	16
7.1	Первый запуск	16
7.2	Работа с лентой	17
7.3	Обновление данных	17
7.4	Лайки сообщений	17
7.5	Настройки темы	17
7.6	Оффлайн режим	17
7.7	Уведомления	17
7.8	Типичные проблемы	17
8	Функциональные модули	18
8.1	Загрузка сообщений	18
8.2	Оффлайн доступ	18
8.3	Лайки сообщений	18
8.4	Фоновая синхронизация	18
8.5	Уведомления	19
8.6	Обработка ошибок	19
9	Алгоритмы и логика работы	19
9.1	Алгоритм обновления ленты	19
9.2	Алгоритм лайка	19
9.3	Алгоритм фоновой синхронизации	20

9.4	Алгоритм оффлайн режима	20
9.5	Формат данных API	20
10	Сетевое взаимодействие и обработка ошибок	21
10.1	Подключение к API	21
10.2	Обработка сетевых ошибок	21
10.3	Стратегия повторов	21
10.4	Валидация и преобразование данных	21
11	Описание реализации	21
11.1	Структура проекта	21
11.2	Сетевые компоненты	22
11.3	Работа с Room	22
11.4	Фоновые задачи	22
11.5	Настройки и предпочтения	23
11.6	Класс MessageEntity	23
11.7	Repository	23
11.8	ViewModel	23
11.9	Adapter и DiffUtil	23
11.10	Мониторинг сети	24
11.11	Темы и стили	24
12	Описание ключевых классов	24
12.1	NewsFragment	24
12.2	NewsViewModel	24
12.3	MessageRepository	24
12.4	MessageDao	24
12.5	SyncWorker	25
12.6	NotificationUtils	25
12.7	SettingsViewModel и SettingsFragment	25
12.8	AppPreferences	25
13	Инструкция по сборке и запуску	25
13.1	Требования к окружению	25
13.2	Запуск проекта	25

13.3	Проверка основных сценариев	26
14	Фоновые процессы и системные события	26
14.1	WorkManager	26
14.2	Уведомления	26
14.3	Реакция на онлайн/оффлайн	26
14.4	Запрос разрешений	26
15	Тестирование	27
15.1	Ручное тестирование	27
15.2	Негативные сценарии	27
15.3	Таблица тест-кейсов	28
15.4	Результаты	28
16	Рекомендации по расширению	28
16.1	Поддержка отправки сообщений	28
16.2	Реальные пользовательские профили	28
16.3	Расширенные уведомления	28
16.4	Синхронизация лайков	29
16.5	Кэширование изображений	29
16.6	Push-уведомления	29
17	Глоссарий	29
18	Заключение	29

ВВЕДЕНИЕ

В рамках курсовой работы был разработан мобильный клиент на платформе Android. Приложение предназначено для отображения ленты новостей, работы в офлайн-режиме, фоновой синхронизации и отправки системных уведомлений о новых данных. Проект опирается на практические результаты лабораторных работ и демонстрирует полный цикл: проектирование, реализация, интеграция с сетью, хранение данных, UI, фоновые процессы.

Цель курсовой работы — спроектировать и реализовать Android-приложение с современной архитектурой и корректной работой в условиях нестабильной сети. В ходе выполнения были решены следующие задачи:

- спроектирована архитектура MVVM с единым репозиторием данных;
- реализована загрузка новостей через сеть и сохранение в локальную базу;
- создан пользовательский интерфейс ленты с Material Design компонентами;
- добавлены интерактивные элементы (лайки, обновление);
- обеспечена фоновая синхронизация с уведомлениями;
- реализована реакция на системные события (онлайн/офлайн);

1 ОБЗОР ПРОЕКТА

1.1 Назначение приложения

Приложение предназначено для демонстрации работы с сетевыми данными и локальным хранилищем в Android. Оно обеспечивает:

- загрузку списка новостей из публичного API;
- работу без сети за счёт кэширования данных в Room;
- интерактивность (лайки сообщений);
- ручное обновление и фоновую синхронизацию;
- уведомления о получении новых данных.

1.2 Контекст и ограничения

Приложение использует публичный REST API (JSONPlaceholder), не поддерживающий реальную отправку сообщений. Поэтому сценарии ориентированы на чтение и синхронизацию. Лайки хранятся локально и не синхронизируются с сервером.

1.3 Целевая аудитория

Проект ориентирован на студентов и начинающих разработчиков, изучающих Android-разработку и принципы современной архитектуры. Также приложение может служить демонстрационным примером для учебных занятий по работе с сетью, БД и UI.

1.4 Ожидаемые результаты

По итогам выполнения работы ожидается:

- работоспособное приложение с полной цепочкой “сеть — база — UI”;
- реализация требований лабораторной работы по UI и фоновой синхронизации;
- формирование навыков архитектурного проектирования;
- демонстрация практики работы с разрешениями и системными событиями.

1.5 Технологический стек

- Kotlin, Android SDK;
- MVVM, LiveData;
- Retrofit + Coroutines для сети;
- Room для локального хранилища;
- WorkManager для фоновой синхронизации;
- Material Design компоненты (AppBarLayout, CardView, FAB);
- Notification API.

2 ТРЕБОВАНИЯ И ПОСТАНОВКА ЗАДАЧИ

2.1 Функциональные требования

В ходе постановки задачи были сформулированы ключевые пользовательские и системные функции:

- загрузка списка сообщений из внешнего API;
- отображение ленты с возможностью ручного обновления;
- сохранение сообщений в локальную базу для офлайн-доступа;
- интерактивность списка (лайки с сохранением состояния);
- фоновая синхронизация с уведомлением о результатах;
- реакция на изменение сетевого состояния;
- наличие настроек темы и офлайн режима.

2.2 Нефункциональные требования

Для обеспечения качества и устойчивости были определены требования:

- невыполнение сетевых операций на UI-потоке;
- устойчивость при отсутствии сети;
- корректная работа при смене конфигурации устройства;
- единая точка доступа к данным через репозиторий;
- корректная работа с разрешениями (уведомления, контакты);
- соответствие гайдлайнам Material Design.

2.3 Ограничения и допущения

В качестве источника данных используется JSONPlaceholder. Он предоставляет статичный набор постов и не поддерживает реальные операции создания сообщений. Поэтому лайки реализованы локально, а сценарий “отправки” не включён в текущую версию проекта.

2.4 Сценарии использования

Основные сценарии работы пользователя:

1. Запуск приложения и просмотр ленты сообщений.
2. Нажатие кнопки “Обновить” для ручной синхронизации.
3. Включение офлайн режима в настройках.
4. Просмотр сохранённых сообщений без сети.
5. Лайк сообщения в списке.
6. Получение уведомления о новых данных в фоне.

3 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

3.1 Типовые приложения класса

Мессенджеры и новостные ленты — один из самых распространённых типов мобильных приложений. Для них критически важны:

- скорость загрузки данных;
- понятная визуальная структура списка;
- устойчивость к потере сети;
- возможность фонового обновления.

Даже в учебном проекте важно воспроизвести эти свойства, так как они отражают реальные ожидания пользователя.

3.2 Пользовательские ожидания

Пользователь ожидает, что данные обновляются быстро и интерфейс остаётся отзывчивым. При отсутствии сети приложение должно корректно объяснить ситуацию, не приводя к ошибкам и зависаниям. Важно также сохранение пользовательских действий (например, лайков), чтобы они не исчезали после обновления.

3.3 Выводы для проектирования

Из анализа следует необходимость:

- локального кэша (Room);
- фона для синхронизации (WorkManager);
- реактивного UI (LiveData);
- минимальной связности между слоями (MVVM).

4 АРХИТЕКТУРА

4.1 Общая схема

Архитектура приложения построена по паттерну MVVM (Model–View–ViewModel). Данный подход обеспечивает разделение ответственности между слоями UI, логики и данных, упрощает поддержку и тестирование.

В проекте выделены следующие слои:

- **UI слой:** фрагменты, адаптеры, layouts;
- **ViewModel:** хранение состояния, запуск операций, обработка ошибок;
- **Repository:** единая точка доступа к данным;
- **Data слой:** Retrofit API, Room Database, DAO.

4.2 UI слой

UI слой состоит из трёх фрагментов: лента, профиль и настройки. Каждый фрагмент отвечает исключительно за отображение состояния, подписывается на LiveData и передаёт пользовательские события в ViewModel. Такой подход снижает связность и облегчает обновление интерфейса.

4.3 ViewModel слой

ViewModel хранит состояние и управляет бизнес-операциями. Например, `NewsViewModel` содержит список сообщений, состояние загрузки и ошибки. Данные при этом остаются доступными при смене конфигурации.

4.4 Repository слой

Репозиторий объединяет источники данных: сеть и локальную базу. В текущем проекте реализован единый класс `MessageRepository`, который инкапсулирует загрузку, преобразование и сохранение данных.

4.5 Data слой

Data слой реализован через `Retrofit` (сеть) и `Room` (локальная БД). DAO предоставляет реактивные потоки, а `Retrofit` выполняет сетевые запросы в корутинах.

4.6 MVVM и поток данных

Состояние ленты сообщений хранится в `NewsViewModel`. `ViewModel` подписывается на поток данных из репозитория и передаёт их в UI через `LiveData`. При изменении данных UI автоматически обновляется. Такой подход обеспечивает реактивность интерфейса и корректное восстановление состояния при смене конфигурации.

Пример привязки состояния к интерфейсу:

```
1 viewModel.messages.observe(viewLifecycleOwner) { items ->
2     adapter.submitList(items)
3     emptyView.visibility = if (items.isNullOrEmpty()) View.VISIBLE else
      View.GONE
4 }
```

Листинг 1 – Привязка `LiveData` к обновлению списка

4.7 Сетевой слой

Сетевой доступ реализован через `Retrofit` с `Gson`-конвертером. API-интерфейс объявлен в `MessageApi`, а клиент создаётся в `ApiClient`. Запросы выполняются в корутинах, что обеспечивает неблокирующую работу UI.

Фрагмент API-интерфейса:

```
1 interface MessageApi {
2     @GET("posts")
3     suspend fun getMessages(): List<MessageDto>
4 }
```

Листинг 2 – Интерфейс `Retrofit` API для сообщений

4.8 Локальное хранилище

Для хранения сообщений используется Room. Сущность `MessageEntity` содержит идентификатор, автора, текст и состояние лайка. DAO предоставляет методы:

- получение списка сообщений как `Flow`;
- вставку/обновление списка;
- обновление лайка;
- полную замену данных.

Пример сущности и DAO:

```
1 @Entity(tableName = "messages")
2 data class MessageEntity(
3     @PrimaryKey val id: Int,
4     val title: String,
5     val body: String,
6     val author: String,
7     val isLiked: Boolean = false
8 )
9
10 @Query("UPDATE messages SET isLiked = :liked WHERE id = :id")
11 suspend fun updateLike(id: Int, liked: Boolean)
```

Листинг 3 – Сущность Room и метод обновления лайка

4.9 Репозиторий как единая точка доступа

Класс `MessageRepository` отвечает за синхронизацию данных. Он:

- загружает список сообщений из сети;
- сохраняет их в базу;
- предоставляет поток данных из Room для UI;
- сохраняет локальные лайки при повторной синхронизации.

Пример обновления данных в репозитории:

```
1 val items = api.getMessages().map { dto ->
2     MessageEntity(
3         id = dto.id,
4         title = dto.title,
5         body = dto.body,
6         author = "User ${dto.userId}",
7         isLiked = likedMap[dto.id] ?: false
```

```

8     )
9 }
10 dao.replaceAll(items)

```

Листинг 4 – Преобразование DTO в сущности и сохранение в Room

4.10 Фоновая синхронизация

Фоновая синхронизация организована через WorkManager. Периодическая задача запускается каждые 15 минут при наличии сети. При успешной синхронизации показывается уведомление пользователю.

Пример создания периодической задачи:

```

1 val request = PeriodicWorkRequestBuilder<SyncWorker>(
2     15, TimeUnit.MINUTES
3 ).setConstraints(
4     Constraints.Builder()
5         .setRequiredNetworkType(NetworkType.CONNECTED)
6         .build()
7 ).build()

```

Листинг 5 – Создание периодической задачи WorkManager

4.11 Уведомления

Уведомления реализованы через Notification API. Создаётся канал уведомлений, после чего при успешной синхронизации отправляется системное сообщение «Новые данные получены».

Фрагмент формирования уведомления:

```

1 NotificationCompat.Builder(context, CHANNEL_ID)
2     .setSmallIcon(R.drawable.ic_launcher)
3     .setContentTitleСинхронизация("")
4     .setContentTextНовые(" данные получены")
5     .build()

```

Листинг 6 – Формирование уведомления о синхронизации

4.12 Онлайн/оффлайн режим

Приложение реагирует на сетевые события через `ConnectivityManager.NetworkCallback`. Также добавлен ручной оффлайн режим, который сохраняется в `SharedPreferences`. В оффлайн режиме:

- отключается ручная синхронизация;
- `WorkManager` не выполняет сетевые запросы;
- пользователь получает уведомление о включении/выключении режима.

5 ПРОЕКТИРОВАНИЕ И МОДЕЛИРОВАНИЕ

5.1 Последовательность обновления

Обновление ленты включает следующие шаги:

1. запрос списка сообщений через `Retrofit`;
2. преобразование DTO в сущности базы данных;
3. объединение с локальными лайками;
4. сохранение обновлённых данных в `Room`;
5. уведомление UI через `Flow/LiveData`.

5.2 Локальные состояния

Состояния, не требующие синхронизации с сервером, хранятся локально. Это включает:

- лайки сообщений;
- выбор темы оформления;
- включённый оффлайн режим.

Такой подход обеспечивает отзывчивость и простоту реализации, при этом сохраняет пользовательские действия между перезапусками приложения.

6 ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

6.1 Навигация

Навигация реализована через `Bottom Navigation` и `Navigation Graph`. Три основные вкладки:

- Лента;
- Профиль;
- Настройки.

6.2 Компоненты Material Design

Для обеспечения единообразного и современного внешнего вида применены компоненты Material Design:

- `AppBarLayout` и `MaterialToolbar` для верхней панели;
- `MaterialCardView` для элементов списка;
- `ExtendedFloatingActionButton` для обновления;
- `TextInputLayout` для ввода данных профиля;
- `SwitchMaterial` для настроек.

Такой набор компонентов обеспечивает согласованную типографику, отступы и поведение на разных версиях Android.

6.3 Темизация и стили

В проекте определены собственные цвета (`colorPrimary`, `textPrimary`) и стили типографики. Для светлой и тёмной темы используются отдельные наборы цветов. Это позволяет корректно отображать контент в обоих режимах и повышает читабельность.

Пример стиля заголовка:

```
1 <style name="TextAppearance.Lab1.Title"
2     parent="TextAppearance.MaterialComponents.Headline6">
3     <item name="android:textColor">@color/textPrimary</item>
4 </style>
```

Листинг 7 – Стиль заголовка для Material Design

6.4 Лента сообщений

Лента реализована на базе `RecyclerView`. Каждый элемент содержит:

- аватар (псевдосгенерированный по первой букве имени);
- имя автора;
- текст сообщения;
- иконку «лайк».

Сверху размещён `AppBarLayout` с заголовком, внизу — `Floating Action Button` для обновления.

6.5 Карточка сообщения

Визуальное представление сообщения основано на `MaterialCardView`. Карточка обеспечивает визуальное отделение элементов и улучшает читаемость списка.

Фрагмент разметки карточки:

```
1 <MaterialCardView ...>
2   <TextView android:id="@+id/message_avatar" .../>
3   <TextView android:id="@+id/message_author" .../>
4   <TextView android:id="@+id/message_body" .../>
5   <ImageButton android:id="@+id/message_like" .../>
6 </MaterialCardView>
```

Листинг 8 – Разметка карточки сообщения

Для аватара используется круглая подложка, а первая буква имени помещается в центр. Такой подход даёт простую визуальную идентификацию без необходимости загрузки изображений.

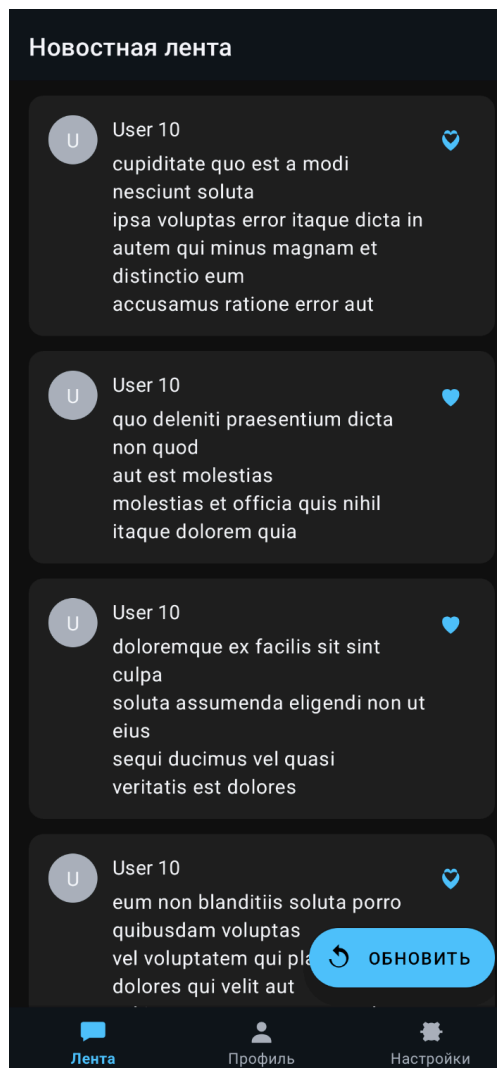


Рисунок 1 – Экран ленты сообщений

6.6 Профиль

Экран профиля позволяет редактировать имя пользователя и статус. Ввод осуществляется через `TextInputLayout` и `TextInputEditText`. Данные сохраняются во `ViewModel`, что позволяет сохранить состояние при повороте экрана.

Для каждого поля используется встроенная в `Material Design` подсказка (`hint`), благодаря чему интерфейс остаётся чистым и компактным. Текстовые поля поддерживают многострочный ввод и автокоррекцию.

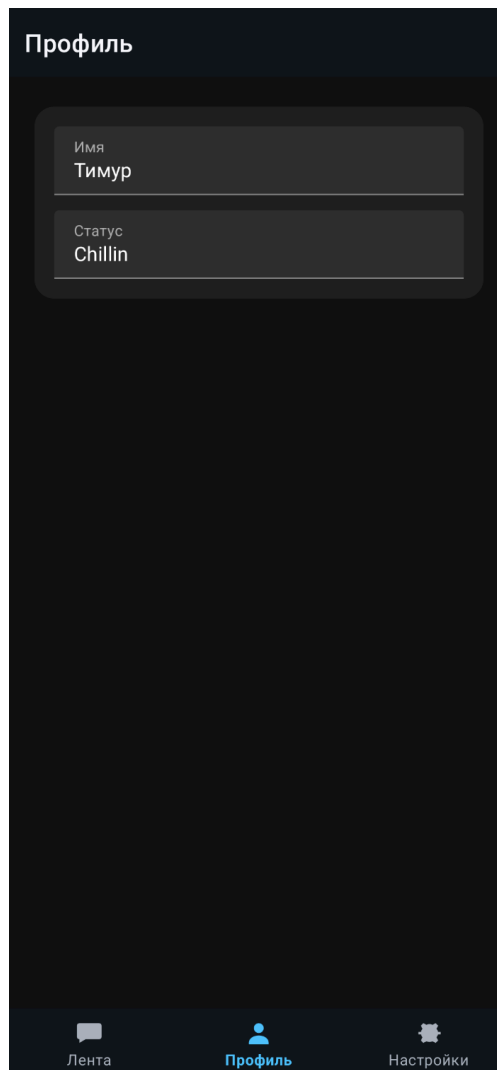


Рисунок 2 – Экран профиля

6.7 Настройки

Экран настроек содержит переключатели темы и оффлайн режима. Переключатель темы меняет `Day/Night` режим. Переключатель оффлайн режима блокирует сетевую синхронизацию.

Пример переключения темы:


```
1 AppCompatActivity.setDefaultNightMode(  
2     if (isChecked) MODE_NIGHT_YES else MODE_NIGHT_NO  
3 )
```

Листинг 9 – Переключение светлой и тёмной темы

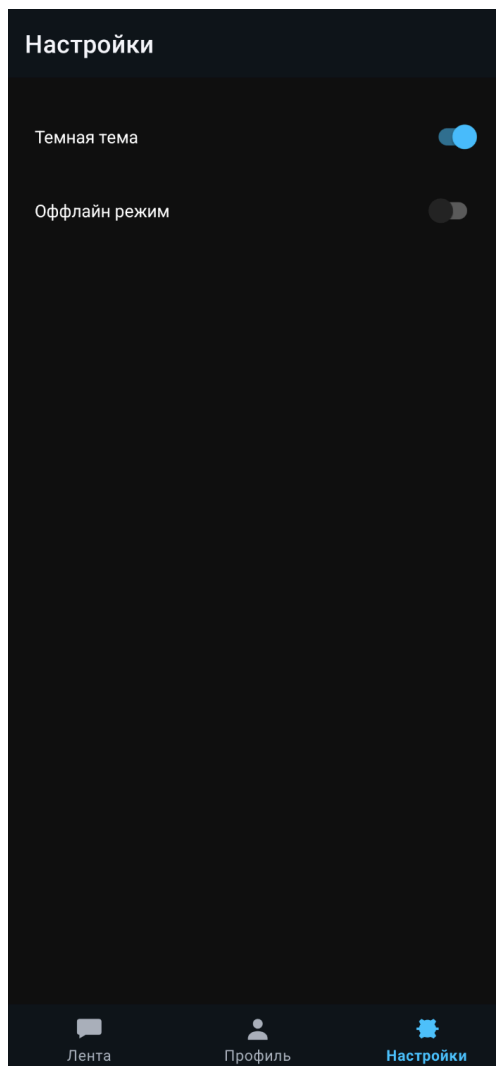


Рисунок 3 – Экран настроек

7 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

7.1 Первый запуск

После установки приложения пользователь попадает на экран ленты. При наличии сети выполняется первичная синхронизация. Если сеть отсутствует, отображается сообщение о работе в офлайн режиме, а лента заполняется данными из локального кэша.

7.2 Работа с лентой

Лента отображает список сообщений. Для просмотра содержимого достаточно пролистывать экран вниз. Заголовок ленты расположен в верхней панели.

7.3 Обновление данных

Для ручного обновления используется кнопка “Обновить” в правом нижнем углу. При нажатии выполняется синхронизация с сетью. Если сеть недоступна, приложение уведомляет пользователя и продолжает показывать локальные данные.

7.4 Лайки сообщений

Чтобы отметить сообщение, нужно нажать на иконку “сердце”. Иконка изменяет состояние и сохраняется даже после перезапуска приложения.

7.5 Настройки темы

В настройках доступен переключатель тёмной темы. Включение или выключение темы применяется сразу и влияет на всю цветовую схему интерфейса.

7.6 Оффлайн режим

Переключатель “Оффлайн режим” отключает синхронизацию, даже если сеть доступна. Это удобно для демонстрации работы без интернета или экономии трафика.

7.7 Уведомления

При успешной фоновой синхронизации приложение отправляет системное уведомление. В Android 13+ необходимо разрешение на уведомления, которое запрашивается при первом запуске.

7.8 Типичные проблемы

- Нет сообщений — проверьте наличие сети и нажмите “Обновить”.
- Не приходят уведомления — убедитесь, что разрешение на уведомления выдано.
- Лента не обновляется — проверьте, не включён ли оффлайн режим.

8 ФУНКЦИОНАЛЬНЫЕ МОДУЛИ

8.1 Загрузка сообщений

При запуске приложения выполняется первичная синхронизация. В дальнейшем пользователь может вручную запустить обновление через FAB. В случае отсутствия сети используется локальный кэш.

Запросы выполняются в корутинах, что предотвращает блокировку UI и обеспечивает плавную работу интерфейса. При ошибках сети формируется информативное сообщение и сохраняется возможность просмотра ранее загруженных данных.

8.2 Оффлайн доступ

Локальная база Room хранит сообщения и лайки. Если сеть недоступна, приложение продолжает показывать данные из базы. Это обеспечивает стабильную работу и предсказуемое поведение.

Оффлайн режим также может быть включён вручную через настройки. В этом случае сетевые запросы блокируются даже при наличии соединения, что полезно для демонстрации режима работы без сети.

8.3 Лайки сообщений

Лайк реализован как интерактивная иконка. Состояние сохраняется в базе данных, что делает его устойчивым к перезапуску приложения.

Пример обработки лайка:

```
1 fun toggleLike(message: MessageEntity) {  
2     viewModelScope.launch {  
3         repository.toggleLike(message)  
4     }  
5 }
```

Листинг 10 – Обработка нажатия лайка в ViewModel

8.4 Фоновая синхронизация

WorkManager запускает синхронизацию по расписанию. Это позволяет обновлять данные в фоне и уведомлять пользователя без необходимости ручного открытия приложения.

Фоновая задача учитывает сетевые ограничения и не выполняется в оффлайн режиме. Это уменьшает лишнюю нагрузку на сеть и делает поведение приложения предсказуемым.

8.5 Уведомления

После успешной синхронизации система отправляет уведомление «Новые данные получены». Это повышает информированность пользователя и демонстрирует работу фоновых задач.

На Android 13 и выше уведомления требуют дополнительного разрешения, которое запрашивается при запуске приложения.

8.6 Обработка ошибок

Ошибки сетевых запросов отображаются через Toast. Сообщения включают состояния «Нет сети» и «Оффлайн режим включен».

9 АЛГОРИТМЫ И ЛОГИКА РАБОТЫ

9.1 Алгоритм обновления ленты

Алгоритм обновления включает проверку режима сети, запрос данных и запись в базу. Логика реализована в ViewModel и репозитории.

Псевдокод процесса обновления:

```
1 if offlineMode:
2     show "Оффлайн" режим включен"
3     exit
4 fetch messages from API
5 map dto -> entity
6 merge likes with local data
7 replace all messages in Room
8 notify UI
```

Листинг 11 – Псевдокод обновления ленты

9.2 Алгоритм лайка

Лайк сообщения меняет локальное состояние иконки и сохраняется в Room. При следующей синхронизации значение не теряется, так как репозиторий переносит флаг лайка.

9.3 Алгоритм фоновой синхронизации

Фоновая задача запускается периодически. Она выполняет синхронизацию при наличии сети и отправляет уведомление.

Псевдокод фонового обновления:

```
1 if offlineMode:
2     finish worker
3 refresh repository
4 if success:
5     show notification
6 else:
7     retry later
```

Листинг 12 – Псевдокод фоновой синхронизации

9.4 Алгоритм оффлайн режима

Ручной оффлайн режим управляется переключателем в настройках и сохраняется в SharedPreferences. При включении:

- обновление через кнопку блокируется;
- WorkManager не выполняет сетевые запросы;
- пользователю показывается тост с текущим состоянием.

9.5 Формат данных API

API возвращает список JSON-объектов со стандартными полями. Пример ответа:

```
1 {
2     "userId": 1,
3     "id": 1,
4     "title": "sunt aut facere...",
5     "body": "quia et suscipit..."
6 }
```

Листинг 13 – Пример ответа API в формате JSON

10 СЕТЕВОЕ ВЗАИМОДЕЙСТВИЕ И ОБРАБОТКА ОШИБОК

10.1 Подключение к API

Подключение выполняется к публичному REST API. Базовый URL задан в клиенте Retrofit. Такой подход позволяет легко заменить источник данных при необходимости.

10.2 Обработка сетевых ошибок

При невозможности получить данные приложение не завершает работу, а переходит на локальный кэш. Пользователь получает понятное сообщение о состоянии сети. Это соответствует принципу “graceful degradation” — ухудшение функционала без полной потери работоспособности.

10.3 Стратегия повторов

В фоновом режиме используется стратегия `Result.retry()` для `WorkManager`, что обеспечивает повторную попытку синхронизации при временных сбоях. В ручном режиме обновление запускается по требованию пользователя.

10.4 Валидация и преобразование данных

DTO из сети преобразуются в сущности базы данных. Поля приводятся к локальной модели, а дополнительные значения (например, имя автора) формируются на стороне клиента. Такой подход обеспечивает единый формат данных во всех слоях приложения.

Дополнительно реализована фильтрация лишних уведомлений о состоянии сети при смене темы и повторной инициализации `Activity`.

11 ОПИСАНИЕ РЕАЛИЗАЦИИ

11.1 Структура проекта

Проект организован по папкам:

- `ui`: фрагменты и адаптеры;
- `data`: `Room`, `DAO`, сущности;
- `network`: Retrofit API;
- `repository`: единая точка доступа;
- `worker`: фоновые задачи.

Данная структура облегчает навигацию по коду и позволяет изолировать изменения в пределах одной подсистемы. Например, изменение API не требует правок в UI, а изменение отображения не затрагивает репозиторий.

11.2 Сетевые компоненты

Сетевой слой включает интерфейс `MessageApi` и клиент `ApiClient`. Клиент настроен на базовый URL и Gson-конвертер.

Пример создания клиента:

```
1 Retrofit.Builder()
2     .baseUrl("https://jsonplaceholder.typicode.com/")
3     .addConverterFactory(GsonConverterFactory.create())
4     .build()
```

Листинг 14 – Создание клиента Retrofit

11.3 Работа с Room

Room используется как локальный кэш. Для миграций применён режим `fallbackToDestructiveMigration`, что упрощает развитие проекта на учебном этапе.

Пример инициализации базы:

```
1 Room.databaseBuilder(
2     context.applicationContext,
3     AppDatabase::class.java,
4     "messenger.db"
5 ).fallbackToDestructiveMigration().build()
```

Листинг 15 – Инициализация базы данных Room

11.4 Фоновые задачи

Фоновая синхронизация реализована через класс `SyncWorker`. Он использует репозиторий для обновления данных и отправляет уведомление при успехе.

Пример выполнения фоновой задачи:

```
1 override suspend fun doWork(): Result {
2     val result = repository.refresh()
3     if (result.isSuccess) {
4         NotificationUtils.showSyncNotification(applicationContext)
5         return Result.success()
6     }
7     return Result.retry()
8 }
```

11.5 Настройки и предпочтения

Настройки пользователя сохраняются в `SharedPreferences`. Это позволяет хранить состояние оффлайн режима и корректно восстанавливать его при перезапуске.

Пример чтения настройки:

```
1 context.getSharedPreferences("app_prefs", MODE_PRIVATE)
2     .getBoolean("offline_mode", false)
```

Листинг 17 – Чтение настройки оффлайн режима

11.6 Класс `MessageEntity`

Сущность содержит поля идентификатора, автора, текста и признака лайка. Эти поля достаточно универсальны для демонстрации требований лабораторной работы и не зависят от конкретного API.

11.7 `Repository`

Репозиторий выполняет синхронизацию и кэширование. Перед сохранением данных учитывается таблица лайков, что предотвращает потерю пользовательских действий при обновлении.

11.8 `ViewModel`

`NewsViewModel` отвечает за обновление данных, обработку ошибок и хранение состояния ленты. `SettingsViewModel` управляет темой и оффлайн режимом.

11.9 `Adapter` и `DiffUtil`

Для отображения списка используется `ListAdapter` с `DiffUtil`, что позволяет эффективно обновлять элементы без перерисовки всего списка. Это особенно важно при больших объёмах данных.

Пример `DiffUtil`:

```
1 override fun areItemsTheSame(old: MessageEntity, new: MessageEntity) =
2     old.id == new.id
3
```



```
4 override fun areContentsTheSame(old: MessageEntity, new: MessageEntity) =  
5     old == new
```

Листинг 18 – Сравнение элементов в DiffUtil

11.10 Мониторинг сети

Сетевой статус отслеживается через `ConnectivityManager.NetworkCallback`. Это позволяет реагировать на изменения соединения и информировать пользователя. Дополнительно реализована фильтрация первого события, чтобы избежать ложных уведомлений при пересоздании Activity.

11.11 Темы и стили

Поддержка светлой и тёмной темы реализована через ресурсы `values` и `values-night`. Переключение выполняется через `AppCompatDelegate`. Цвета и типографика вынесены в отдельные ресурсы, что упрощает масштабирование дизайна.

12 ОПИСАНИЕ КЛЮЧЕВЫХ КЛАССОВ

12.1 NewsFragment

Фрагмент ленты отвечает за отображение списка сообщений, запуск ручного обновления и реакцию на ошибки. Здесь создаётся адаптер и настраивается RecyclerView.

12.2 NewsViewModel

ViewModel управляет состоянием ленты и запускает синхронизацию. Она хранит LiveData со списком сообщений и ошибки, а также предоставляет метод переключения лайков.

12.3 MessageRepository

Репозиторий объединяет сеть и Room. В нём реализована логика сохранения локальных лайков при обновлении, что предотвращает потерю пользовательских действий.

12.4 MessageDao

DAO предоставляет методы чтения списка сообщений, вставки и обновления состояния лайков. Использование Flow обеспечивает реактивное обновление UI.

12.5 SyncWorker

Фоновая задача, выполняемая WorkManager. Она учитывает ограничения сети и оффлайн режим, а при успехе отправляет уведомление.

12.6 NotificationUtils

Вспомогательный класс для создания канала уведомлений и отправки сообщений о синхронизации. Вынесение в отдельный класс снижает связность и упрощает поддержку.

12.7 SettingsViewModel и SettingsFragment

Класс настроек управляет переключателями темы и оффлайн режима. Состояние сохраняется в SharedPreferences. Экран настроек обеспечивает интерактивность и обратную связь через тосты.

12.8 AppPreferences

Класс-обёртка над SharedPreferences. Реализует простые методы чтения и записи режима оффлайн, скрывая детали хранения.

13 ИНСТРУКЦИЯ ПО СБОРКЕ И ЗАПУСКУ

13.1 Требования к окружению

Для сборки проекта требуется Android Studio и установленный Android SDK (API 24+). Рекомендуется использовать эмулятор, соответствующий актуальному API уровню, либо физическое устройство.

13.2 Запуск проекта

Порядок запуска приложения:

1. Открыть каталог проекта в Android Studio.
2. Дождаться завершения Gradle синхронизации.
3. Выбрать устройство или эмулятор.
4. Запустить приложение через кнопку Run.

При необходимости сборки APK можно использовать Gradle-команду:

```
1 ./gradlew assembleDebug
```

Листинг 19 – Сборка debug APK через Gradle

13.3 Проверка основных сценариев

После запуска рекомендуется проверить:

- наличие данных в ленте после первого обновления;
- корректность работы оффлайн режима;
- сохранение лайков после перезапуска;
- появление уведомления при фоновой синхронизации.

14 ФОНОВЫЕ ПРОЦЕССЫ И СИСТЕМНЫЕ СОБЫТИЯ

14.1 WorkManager

WorkManager используется для периодической синхронизации, так как он гарантирует корректное выполнение задач в фоне и учитывает ограничения сети. Интервал выбран 15 минут — минимально допустимый для периодической работы.

Также учитывается оффлайн режим. Если пользователь включил ручное отключение сети, задача завершится без выполнения сетевого запроса.

14.2 Уведомления

При успешной синхронизации создаётся уведомление. В Android 13+ требуется разрешение POST_NOTIFICATIONS, поэтому оно запрашивается при запуске приложения.

Канал уведомлений создаётся один раз при старте приложения, что соответствует рекомендациям Android.

14.3 Реакция на онлайн/оффлайн

Приложение реагирует на изменение сети через сетевой коллбэк. Для предотвращения лишних уведомлений при смене темы используется фильтрация первого события. Пользователь может вручную включить оффлайн режим через настройки.

14.4 Запрос разрешений

При первом запуске приложение запрашивает разрешение на уведомления и доступ к контактам. Запрос выполняется централизованно в главной активности.

Пример запроса разрешения:

```
1 ActivityCompat.requestPermissions(  
2     this,  
3     arrayOf(Manifest.permission.POST_NOTIFICATIONS),  
4     1001
```

15 ТЕСТИРОВАНИЕ

15.1 Ручное тестирование

Проверены следующие сценарии:

- запуск приложения и первичная загрузка сообщений;
- включение/выключение оффлайн режима;
- работа без сети (данные из Room);
- ручное обновление списка;
- фоновые уведомления о синхронизации;
- корректность сохранения лайков.

15.2 Негативные сценарии

Дополнительно проверены случаи:

- отсутствие сети при запуске приложения;
- отказ в разрешении на уведомления;
- повторное включение оффлайн режима;
- быстрые последовательные нажатия на “Обновить”.

15.3 Таблица тест-кейсов

Сценарий	Описание	Результат
Загрузка ленты	При старте выполняется синхронизация и отображается список	Успешно
Оффлайн режим	Включение оффлайн режима блокирует сеть	Успешно
Лайк сообщения	Нажатие на иконку изменяет состояние и сохраняется	Успешно
Нет сети	При отсутствии сети показывается кэш и сообщение	Успешно

15.4 Результаты

Все основные сценарии работают корректно. Приложение устойчиво к смене ориентации экрана и к отсутствию сети.

16 РЕКОМЕНДАЦИИ ПО РАСШИРЕНИЮ

16.1 Поддержка отправки сообщений

При наличии API с поддержкой POST запросов можно добавить форму отправки сообщений и расширить функционал приложения.

16.2 Реальные пользовательские профили

Возможна интеграция с системой авторизации и хранение профиля пользователя в сети.

16.3 Расширенные уведомления

Можно добавить группировку уведомлений, отображение количества новых сообщений и быстрые действия.

16.4 Синхронизация лайков

При наличии полноценного API возможно синхронизировать лайки с сервером и поддерживать единый статус на разных устройствах.

16.5 Кэширование изображений

Для реальных аватаров рекомендуется использовать библиотеку для загрузки и кэширования изображений (например, Coil или Glide). Это улучшит UX и снизит нагрузку на сеть.

16.6 Push-уведомления

В качестве дальнейшего развития можно интегрировать push-уведомления (FCM), чтобы получать новые сообщения без периодического опроса API.

17 ГЛОССАРИЙ

- **MVVM** — архитектурный паттерн Model–View–ViewModel.
- **Room** — библиотека для работы с локальными базами данных в Android.
- **Retrofit** — HTTP-клиент для работы с REST API.
- **DAO** — объект доступа к данным (Data Access Object).
- **LiveData** — реактивный контейнер данных с учётом жизненного цикла.
- **WorkManager** — библиотека для фоновых задач.
- **FAB** — Floating Action Button (плавающая кнопка действия).
- **API** — программный интерфейс взаимодействия.

18 ЗАКЛЮЧЕНИЕ

В ходе курсовой работы разработано Android-приложение, демонстрирующее полный цикл работы с сетевыми данными, локальным хранилищем, фоновыми задачами и современным интерфейсом. Применён паттерн MVVM, реализованы репозиторий и реактивное обновление UI. Решение соответствует поставленным требованиям, устойчиво к отсутствию сети и обеспечивает основу для дальнейшего расширения.